Where a CDCL SAT Solver Spends Its Time

Alperen Buğra Başar • Bilkent University • bugra.basar@ug.bilkent.edu.tr

https://github.com/AlperenBgrBsr/SATSolver

I wrote a CDCL SAT solver from scratch in C++17 and measured where its time goes, down to single instructions. Every answer it gives is checked independently: satisfying assignments against the formula, and unsatisfiability through a DRAT proof verified by `drat-trim`.

- **A four-number cost model.** A skip costs 6.0 cycles, a search for a new watched literal 86.8, each literal it checks 2.1, and each conflict 4232. Fitted over 39 runs, these predict total cycles with a median error of 2.5%.
- **A result that did not survive control.** One run made the solver look 12 times faster than MiniSat. Over 50 shuffled copies, the comparison flipped with the signs of the literals, and removing MiniSat's restarts and clause minimization made the two solvers statistically indistinguishable (conflict ratio 1.06, 95% interval spanning 1).
- **Memory, with the search held fixed.** Padding every clause grew the footprint 20 times without changing a single decision. The cost per visit stays flat while the data fits in L2 and rises by 23% beyond it, only in code that reads the padded data.
- **Trust has a price, and it is small.** Proof logging adds 11–23% cycles, and checking a proof costs about as much as solving.

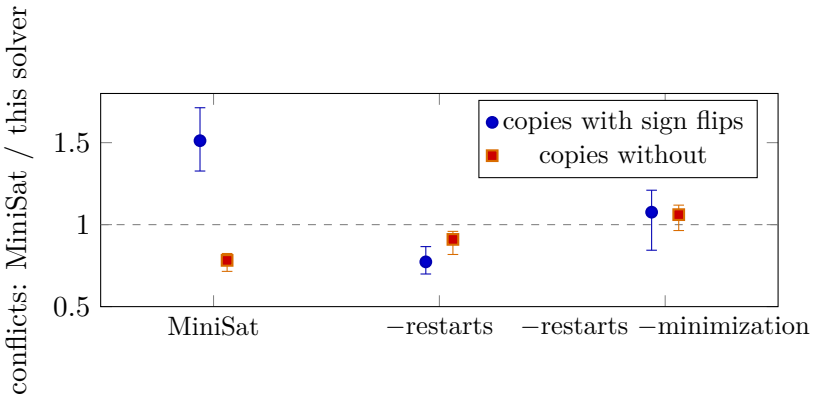


Figure 1: Conflicts of MiniSat divided by conflicts of this solver, median over 50 shuffled copies with 95% intervals, as MiniSat's features are removed.

Limits: one machine, instances of at most 250 variables, and no hardware performance counters, so cache and branch claims are inferred from several independent measurements. The full report, with methods, data and reproduction commands, is in the repository.